

Wine Quality Prediction Report

Introduction:

Our dataset contains information on different variants of the Portuguese “Vinho Verde” wine, and contains eleven features, all physiochemical traits of a given wine (as described in Table 1 featured in the appendix). The outcome variable we aim to predict is the quality of the wine, which is a score from 1 to 10 as determined by sensory assessors in the CVRVV (Viticulture Commission of the Vinho Verde Region). Our goal is to create models that can predict the wine quality based on these eleven features. Being able to predict quality based on these objective measures will be helpful for companies who sell Vinho Verde wine and would like to maximize quality or choose appropriate prices for different qualities of wine.

We chose to separate the white and red wine data into two datasets because in the full dataset, there's a very uneven number of observations between the white wines and red wines: 4898 white wine observations, but less than half that number of red wine observations, only 1599. This significant imbalance in the number of observations could skew the models to be better at predicting the wine quality of white wine, which is not what we wanted. Additionally, we believe that the physicochemical properties are likely fundamentally different between white and red wines as well. Furthermore, we treated wine quality prediction as a regression problem because, as we will cover in the exploratory data analysis section, there are many classes in the response variable if wine quality was treated as a categorical variable, and some classes have much fewer observations, no matter if we looked at white wine or red wine data, so classification methods with several possible categories probably wouldn't classify data into those classes with very high accuracy; additionally, classification models with lower numbers of categories would be less useful to companies aiming to have specific predictions for quality. Regression methods wouldn't run into this issue.

Exploratory Data Analysis:

After splitting the data into separate red wine and white wine datasets (and checked that there were no NA values), we confirmed their respective number of observations and the number of observations for each wine “quality” value are not consistent for all the “quality” values. We then decided to take a closer look at the predictors influencing wine quality. First, we examined the distribution of each predictor using histograms, then assessed their correlation with other variables, and finally identified any potential outliers using boxplots.

From looking at the distributions of the predictors, we saw that most of the predictors were very right-skewed, with the only exception for both datasets being the pH of the wine. One key difference between predictors was that while the red_wine density is approximately normally distributed, white_wine density is extremely right skewed. This may be due to the difference in composition of red and white wine such as more sugar content or other dissolved solids. For our purposes, this might mean that the white_wine dataset may require more flexible modelling techniques, such as transformation or nonlinear methods.

After we plotted the correlation matrix for red and white wine separately, we noticed that the relationship between pH and fixed acidity is more strongly negatively correlated in red wine than with white wine, the relationship between residual sugar and density is much more strongly positive for white wine than for red wine. Additionally there exists strong correlations between many different predictors in both the red and white wine datasets which means that it may be better to use methods that address multicollinearity.

Finally, the boxplots showed us the presence of any outliers for the predictors in both datasets. There were many outliers in most predictors for both of the datasets which indicates that inflexible models may not perform as well due to high variance and sensitivity to outliers. For red wine, alcohol, citric acid, and sulfates appear to have less than 10 outliers, and for white wine, alcohol, residual sugar, and density were the same. This may suggest that these features are stronger predictors of wine quality and may work better in more inflexible models, though overall, flexible models would appear to fit the datasets better.

Methodology:

Keeping the white and red wine datasets separate, to predict wine quality, we repeated the validation set approach 10 times for each machine learning method we used. Each time, we randomly divided the data into two parts: a training set and a validation set, each corresponding to half of the data. We fit a model on the training set (using default parameters unless otherwise specified), and used that model to predict the responses for the observations in the validation set. From those predictions, we got an estimate of the test error: the validation set Mean Squared Error (MSE). Finally, we took the average of the 10 MSEs. We compared these average MSEs across all the machine learning methods we used. To determine if classification models would work for this data, we tested a logistic regression model. We created a new variable that categorizes wine quality as either “low” (≤ 5) or “high” (>5). These categories corresponded to 0

(for “low”) or 1 (for “high”). We used 0.5 as our cutoff for classifying a result as “high quality”. We obtained an estimated test error rate of 25.0% for white wine and 25.5% for red wine. For white wine, 91.3% of misclassifications occurred near the low/high border (true quality of 5 or 6), and for red wine 92.3% of misclassifications occurred at these values. The high degree of misclassifications pertaining to the most common quality ratings support our idea that classification methods may not be useful for these data, given the distribution of ratings for both white and red wine, which is why we focused on regression models instead.

Multiple Linear Regression:

We first fit an ordinary least squares regression model on the data, using all of the predictors. In the model we created for red wine, we obtained a test MSE of 0.429 using the methodology above. In the white wine model, we obtained a test MSE of 0.573. We followed this initial model with an analysis of the MSE when using only a subset of the variables.

Subset Selection:

An alternative to fitting a model using least squares on all the predictors is to choose a subset of predictors that we think are related to the response, then fit a model using least squares on that subset of predictors. We used best subset, forward stepwise, and backward stepwise selection.

Best Subset Selection: Using the `regsubsets` function to build a best subset selection model on the training data, we set the maximum size of subsets to examine as “`nvmax=11`,” because we wanted to have 11 possible models to choose from for the next step, and used the default “method” of exhaustive search. With each of these 11 models, we made predictions on our validation set and determined the smallest MSE. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.5720665 for white wines and 0.4266696 for red wines.

Forward Stepwise Selection: Using the `regsubsets` function to build a forward selection model on the training data, we set the maximum size of subsets to examine as “`nvmax=11`,” because we wanted to have 11 possible models to choose from for the next step, and used “method=“forward”” to specify forward selection. With each of these 11 models, we made predictions on our validation set and determined the smallest MSE. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.5715544 for white wines and 0.4266943 for red wines.

Backward Stepwise Selection: Using the `regsubsets` function to build a backward selection model on the training data, we set the maximum size of subsets to examine as “`nvmax=11`,” because we wanted to have 11 possible models to choose from for the next step, and used “`method=“backward”`” to specify backward selection. With each of these 11 models, we made predictions on our validation set and determined the smallest MSE. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.5720665 for white wines and 0.4269992 for red wines.

Thus, out of the subset selection methods, forward selection produced the smallest average MSE of 0.5715544 for white wines, but only slightly smaller than best subset and backward selection, and best subset selection produced the smallest average MSE of 0.4266696 for red wines, but only slightly smaller than forward and backward selection.

Shrinkage Methods:

An alternative to choosing a subset of predictors is shrinking regression coefficient estimates towards zero, which can reduce their variance and potentially improve model performance. The two techniques we used were ridge regression and lasso regression.

Ridge Regression: Before building the training model, we used 10-fold cross validation to choose the best/smallest lambda value, which controls the strength of the shrink penalty, based on our training data. We then used the `glmnet` function to build a ridge regression model on the training data, by setting “`alpha=0`,” which indicates ridge penalty, and “`lambda`” to be the best/smallest lambda value from above. With this model, we made predictions on our validation set, specifying that parameter “`s`” was the same best/smallest lambda value from above. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.5723425 for white wines and 0.4310294 for red wines.

Lasso Regression: Before building the training model, we used 10-fold cross validation to choose the best/smallest lambda value, which controls the strength of the shrink penalty, based on our training data. We then used the `glmnet` function to build a lasso regression model on the training data, by setting “`alpha=1`,” which indicates lasso penalty, and “`lambda`” to be the best/smallest lambda value from above. With this model, we made predictions on our validation set, specifying that parameter “`s`” was the same best/smallest lambda value from above. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.5723235 for white wines and 0.4314624 for red wines.

Thus, out of the shrinkage methods, ridge and lasso regression performed similarly well, with lasso regression producing a slightly smaller average MSE of 0.5723235 for white wines, and ridge regression producing a slightly smaller average MSE of 0.4310294 for red wines.

Dimension Reduction Methods:

While exploring the wine quality dataset, we noticed potential multicollinearity among the predictors, which could complicate our model's interpretation and performance. So, we decided to try PCR and PLS, as both methods are effective at handling correlated predictors by either reducing dimensionality or extracting components that maximize variance and covariance with the outcome, which would then allow us to identify the most relevant factors influencing wine quality while preventing overfitting.

Principal Components Regression (PCR): For PCR, we first found the best number of components for the model. This was done by fitting a PCR model using `pcr()` to all of the features for every possible number of components. We then selected the optimal number of components after calculating the 10-fold cross validation error for each manually, then fit a final PCR model for that iteration using the optimal component number, after which we used the validation set to calculate the validation MSE. Our resulting average MSEs were 0.4301747 for red wine and 0.5773318 for white wine.

Partial Least Squares (PLS): For PLS, we used a very similar method to determine the average MSE. We first performed a manual 10-fold CV method to determine the optimal number of components for PLS using the `pls()` function to fit the model. The optimal component number was determined using the one associated with the lowest 10-fold CV error. Then, using the optimal number of components, we fit the final PLS model for that iteration using the training data, then used the validation set to find the MSE for that iteration. We then took the average over the 10 iterations, and determined that the validation MSE was 0.4305369 for red wine, and 0.5762615 for white wine.

Nonlinear Methods:

Polynomial Regression: We performed polynomial regression by splitting the data into training and validation sets 10 times. Within each of these iterations, we evaluated the polynomial degrees of 1 to 3 using 10-fold CV to identify the degree that minimized RMSE. After determining the optimal degree, we trained a linear model on the training set and made

predictions on the test set. The MSE was then saved for each iteration and we took the average of the MSE. For red wine, the validation error was 0.4236251, and for white wine it was 0.5683874.

Basis Functions: We applied polynomial basis functions to all predictors in both red and white datasets to model quality. For each dataset, we split the data into a training and validation set. On the training set, we performed polynomial transformations from degrees 1 to 3, then trained a linear model with `train()` for each degree and used a 10-fold CV to determine the best degree for polynomial transformation. We then fit a model on the training data using the optimal degree, making predictions on the validation set and calculating the MSE. This was performed 10 times, after which we took the averages of the validation MSEs. For red wine, the average validation MSE was 0.4222335, and it was 0.569291 for white wine.

Smoothing Splines: For each dataset, within each iteration of the validation-set approach, we performed a 10-fold CV on the training set by fitting smoothing splines using all predictors, trying degrees of freedom from 2 to 10. If a predictor has more unique values than the given df, we use `smooth.spline()` to fit the model, otherwise, we assign the mean quality value, however, this may have introduced bias. We then predict on the validation folds using `predict()`, and calculate MSE. The best df, corresponding to the lowest 10-fold CV error is chosen, and we train the final smoothing model on the training set. We then predicted on the validation set, and outputted the average validation MSE across the 10 iterations. For red wine, the average MSE was 0.5702943, and it was 0.6941715 for white wine.

Generalized Additive Model (GAM): We allowed curved relationships between each individual variable and the outcome by creating smooth terms using the default thin plate regressions splines. After doing ten repetitions of cross validation, we obtained an average validation MSE of 0.5220522 for white wine and 0.426529 for red wine.

Tree-Based Methods:

Tree-based methods can be applied to regression problems, so they're applicable to our task of predicting wine quality.

Regression Tree: After building an initial tree on the training data, we used 10-fold cross validation to choose the best tree size that corresponded to the lowest deviance. We then used cost-complexity pruning to prune the tree, setting the parameter "best" to be the best tree size from above. With this tree, we made predictions on our validation set. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.5838844 for white wines and

0.4858926 for red wines. But we thought the regression tree method would not be robust enough, so we tried aggregating many trees to improve predictive performance, hence the use of bagging, random forest, and boosting to follow this.

Bagging: Using the `randomForest` function to build a bagging model on the training data, we set the number of predictors randomly sampled as candidates at each split as “`mtry=11`” because we use all 11 predictors in bagging. A default parameter of note that we also used was “`ntree=500`,” which is the number of trees we chose to grow. With this model, we made predictions on our validation set. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.4231874 for white wines and 0.3719684 for red wines.

Random Forest: Using the `randomForest` function to build a random forest model on the training data, we set the number of predictors randomly sampled as candidates at each split as “`mtry=3`” because the default value for regression is the floor of $p/3$, so in our case, $p=11$, and doing the computation we get the value 3. A default parameter of note that we also used was “`ntree=500`,” which is the number of trees we chose to grow. With this model, we made predictions on our validation set. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.4169299 for white wines and 0.3720447 for red wines.

Boosting: Using the `gbm` function to build a boosting model on the training data, we set “`distribution=“gaussian”`” because we consider our response variable (wine quality) to be continuous and assume it follows a Gaussian distribution. Some default parameters of note that we also used were “`shrinkage=0.1`,” a reasonable learning rate, and “`n.trees=100`,” the number of trees to fit. We also used the default “`interaction.depth=1`,” meaning the highest level of variable interactions allowed is only 1. With this model, we made predictions on our validation set. Repeating this 10 times, the resulting average MSE from our validation set approach was 0.531613 for white wines and 0.4216841 for red wines.

Thus, out of the tree-based methods, random forest produced the smallest average MSE of 0.4169299 for white wines, but only slightly smaller than with bagging. Bagging produced the smallest average MSE of 0.3719684 for red wines, but only slightly smaller than with random forest.

Best Methods and Conclusion:

Out of all the regression methods we fit to predict wine quality, bagging and random forest performed the best in terms of average MSE using the validation set approach we proposed. To

summarize, the smallest average MSE we got for white wines was 0.4169299, using a random forest model (although the bagging model's performance was comparable), and the smallest average MSE we got for red wines was 0.3719684, using a bagging model (although the random forest model's performance was comparable). Compared to multiple linear regression, our initial, simplest model, there was a significant improvement in average MSE from the original 0.573 for white wines, and also a clear improvement in average MSE from the original 0.429 for red wines. Our findings suggest that the data is better modeled by more complex models than multiple linear regression to capture more complex and nonlinear relationships. As some next steps, we could more carefully tune the parameters in our current models, as well as try other regression methods to help improve our predictions.

In the wine industry, some use cases for the prediction model we built (as prefaced in the Introduction) include the following: after physicochemical tests are conducted, low quality wines can be flagged by our model, helping with quality control, and the wine quality that our model predicts could inform pricing, justifying high prices for higher quality wines. One restriction of our exploration is that we don't have access to certain variables such as grape origin. In the future, we could consider merging data from other datasets that could provide other predictors, such as soil types, microclimates, and vineyard altitude, that might help improve the prediction of wine quality.

Appendix

Table 1: Descriptions of each feature in the dataset.

fixed.acidity	grams of tartaric acid per liter
volatile.acidity	grams of acetic acid per liter
citric.acid	grams of citric acid per liter
residual.sugar	grams of natural grape sugars remaining per liter
chlorides	grams of sodium chloride per liter
free.sulfur.dioxide	milligrams of unbound sulfur dioxide per liter, available to act as an antioxidant/antimicrobial agent
total.sulfur.dioxide	total milligrams of sulfur dioxide per liter
density	density of the wine, in grams per cubic centimeter
pH	pH of the wine
sulfates	grams of potassium sulfate per liter
alcohol	alcohol percentage by volume

Method	Red MSE	White MSE
Ordinary Least Squares Regression	0.4289739	0.5731902
Best Subset Selection	0.4266696	0.5720665
Forward Selection	0.4266943	0.5715544
Backward Selection	0.4269992	0.5720665
Ridge Regression	0.4310294	0.5723425
Lasso Regression	0.4314624	0.5723235

PCR	0.4301747	0.5773318
PLS	0.4305369	0.5762615
Polynomial Regression	0.4236251	0.5683874
Basis Functions	0.4222335	0.569291
Smoothing Splines	0.5702943	0.6941715
GAM	0.426529	0.5220522
Regression Tree	0.4858926	0.5838844
Bagging	0.3719684	0.4231874
Random Forest	0.3720447	0.4169299
Boosting	0.4216841	0.531613